Machine Coding: Drag and Drop Interaction (Sortable Cards)

- **Problem:** Build a list (grid) of cards the user can reorder by dragging one card onto another. Show which card is being dragged and where it will land, clean up correctly when the drag is cancelled, and let keyboard and screen-reader users reorder too.
- **Tech stack:** React 19 + Vite (JavaScript, plain CSS, native HTML5 Drag and Drop, no library)
- **Time limit:** 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Clarify** the requirements. Don't guess; ask.
2. **List the interactions.** Every user action and what it should do becomes your checklist.
3. **Sketch the architecture.** Components, hooks and modules, with one job each.
4. **Design the state.** Find the minimum you have to store; derive the rest.
5. **Define the data and contracts.** Example data and the API shape (mock it if there's no API).
6. **Build the simplest working version first,** then add one working layer at a time.
7. **Handle every state and edge case:** empty, loading, error, success.
8. **Wrap up.** Explain the trade-offs and what you'd add with more time.

Time budget (60 min)

Phase	Time	Output
Requirements + clarifying questions	5 min	List vs grid, mouse vs touch, a11y scope
Use cases + state + data	5 min	`items` with ids, `draggingId`, `overId`
Pure `moveItem` + render list	8 min	Cards on screen, reorder logic tested
HTML5 drag: start / over / drop / end	12 min	Mouse reorder works
Visual feedback (dragging, drop target)	7 min	User sees what is happening
Keyboard reorder + live announcements	13 min	Accessible version
Empty state, polish, test + wrap-up	10 min	Demo + trade-offs

Tip: think out loud

The interviewer is grading how you reason, not just the final code. Say each decision as you make it: *"I'll track the dragged card by its **id**, not its index, because the index changes the moment the list reorders. And I'll reset*

the drag state in `dragend`, not in `drop`, because `drop` never fires when the user lets go outside a card."

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Render a list of cards in a wrapping grid.
- Drag a card with the mouse and drop it on another card: the dragged card takes that card's position and the others shift.
- While dragging: the source card is faded, the card under the pointer shows a drop indicator.
- Dropping outside a card or pressing Escape mid-drag cancels the move and leaves no leftover styles.
- Keyboard: focus a card, **Space/Enter** to pick up, **arrow keys** to move, **Space/Enter** to drop, **Escape** to cancel and restore the original order.
- Screen readers hear what happened ("Item 3 moved to position 1 of 6").

Non-functional requirements

Area	What it means here
Performance	`dragover` fires every ~50 ms; don't re-render on every event when nothing changed. Reorder is O(n).
Scalability	Pure `moveItem` works for any list; with hundreds of cards, add virtualization or a library.
Error handling	Invalid indexes, dropping on itself, drop with no source: no change, no crash.
Accessibility	Keyboard reorder, visible focus, `aria-live` announcements, help text linked by `aria-describedby`.
Reusability	`SortableItem` is presentational; reorder logic is a pure util that any list can reuse.
Extensibility	Items are objects with ids, so adding fields, persistence or multiple lists (Kanban) doesn't change the core.

Clarifying questions to ask the interviewer

1. Is it a single list, or do items move **between** lists (Kanban)?
2. Vertical list or a wrapping grid? (Changes which arrow keys mean "earlier".)
3. Must it work on **touch** devices? Native HTML5 DnD does not fire on most mobile browsers.
4. Can I use a library (dnd-kit, react-beautiful-dnd), or should it be native?
5. Should the new order **persist** (localStorage / API)?
6. Should other cards animate into place while dragging, or is "reorder on drop" enough?

Assumptions we will build with

- One list, displayed as a wrapping grid; order is a single array.
- Native HTML5 Drag and Drop for mouse, no library.

- Reorder happens **on drop** (not live while hovering). No animation of the other cards.
- Full keyboard support and screen-reader announcements are in scope.
- Touch support and persistence are "with more time".

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	6 cards in order, each focusable, help text visible
Mouse down + drag a card	Source card fades (`dragging`), browser shows drag ghost
Drag over another card	That card shows a dashed border (`over`); cursor shows "move"
Drop on another card	Source takes the target's position; styles cleared; announcement
Drop on the same card	No change; styles cleared
Release outside any card / press Esc mid-drag	No change; `dragend` clears styles
Tab to a card	Yellow focus ring
Space / Enter on a focused card	Card is picked up (`grabbed`), announcement with instructions
Arrow Left/Up while grabbed	Card moves one position earlier; focus stays on it
Arrow Right/Down while grabbed	Card moves one position later
Arrow at the first / last position	Nothing happens (clamped)
Escape while grabbed	Order restored to what it was before pick-up
Space / Enter while grabbed	Card dropped, announcement "dropped at position N"
Empty list	"No items to reorder." message instead of the grid

This table is the implementation and test checklist.

3. Design the architecture

Module	Responsibility
`App`	Owns the list order and the drag/keyboard state; all handlers live here
`SortableItem`	Renders one card, wires DOM events to the handlers, picks CSS classes from flags
`utils/reorder.js`	Pure `moveItem(list, from, to)` : returns a new array
`data/items.js`	Initial items with stable ids (mock data)
Live region (`<p aria-live>`)	Announces every move to screen readers

```
App  (items, draggingId, overId, grabbed, announcement)
 ├── <p id="dnd-help">            instructions (aria-describedby target)
 ├── <ul className="container">
 │     └── SortableItem × n        (item, position, flags, handlers)
```

```
  └── <p aria-live="assertive">     announcement
utils/reorder.js  ← used by App
data/items.js     ← initial state
```

Start simple: the first version is just `App` with the list inline. `SortableItem` is split out once the event wiring and class logic make the JSX noisy. A `useSortable` hook is only worth it when a second list needs the same behavior.

4. Design the state / data model

State or field	Purpose
`items`	The order: array of `{ id, title }`. The only thing that really matters.
`draggingId`	Id of the card being mouse-dragged (`null` when idle). Fades the source; is the drop source.
`overId`	Id of the card currently under the pointer. Shows the drop indicator.
`grabbed`	Keyboard mode: `{ id, snapshot }`. `snapshot` is the order before pick-up, for Escape.
`announcement`	Text for the `aria-live` region.

```
const [items, setItems] = useState(INITIAL_ITEMS);
const [draggingId, setDraggingId] = useState(null); // mouse drag source
const [overId, setOverId] = useState(null);         // current drop target
const [grabbed, setGrabbed] = useState(null);       // keyboard: { id, snapshot }
const [announcement, setAnnouncement] = useState("");
```

Senior note

- **Derived, not stored:** a card's position (`index + 1`), `total`, `isDragging`, `isOver`, `isGrabbed` and the CSS class string are computed during render. Storing them would mean keeping them in sync on every move.
- **Ids, not indexes:** the original version stored `dragItem` as an index. Indexes are positions, and positions are exactly what this feature changes. Ids stay true no matter how the list moves.
- **Ownership:** `App` owns everything because drag source, drop target and order must all change together. `SortableItem` owns nothing; it only reports events.
- `draggingId` and `grabbed` are separate on purpose: mouse and keyboard are two different interactions with different cleanup rules.

5. Design the data structures and contracts

Example data

```
export const INITIAL_ITEMS = [
  { id: "item-1", title: "Item 1" },
  { id: "item-2", title: "Item 2" },
```

```
  // ... item-6
];
```

Why ids: the original used the title string as both content and React `key`. Two cards named "Todo" would collide. A stable id gives React a correct key (the DOM node moves with the card, so focus and CSS transitions follow it) and gives drag logic an identity that doesn't change when the order does.

Contracts

```
// utils/reorder.js
moveItem(list: T[], from: number, to: number): T[]
// returns a NEW array; returns the same `list` reference when from === to or an index is out of range
moveItem(["a","b","c","d"], 0, 2)  // → ["b","c","a","d"]
moveItem(["a","b","c","d"], 3, 0)  // → ["d","a","b","c"]
```

```
// SortableItem props
<SortableItem
  item={{ id: "item-3", title: "Item 3" }}
  position={3} total={6}
  isDragging={false} isOver={true} isGrabbed={false}
  handlers={{ onDragStart, onDragOver, onDrop, onDragEnd, onKeyDown }}
/>
```

```
// Drag payload (HTML5 dataTransfer)
e.dataTransfer.setData("text/plain", "item-3");
e.dataTransfer.effectAllowed = "move";
```

No real API. If the order had to be saved: `PATCH /lists/:id/order` with body `{ "order": ["item-2","item-3","item-1", ...] }`, sent after the drop (optimistic update, roll back on error).

6. Trace the data flow

Mouse: drag "Item 1" onto "Item 4".

```
mousedown + move on Item 1
   ↓  browser fires dragstart on Item 1
onDragStart(e, "item-1")
   ↓  dataTransfer.setData("text/plain", "item-1"); setDraggingId("item-1")
render: Item 1 gets class "dragging" (faded)
   ↓  pointer moves over Item 4 → dragover fires repeatedly
onDragOver(e, "item-4")
   ↓  e.preventDefault()  (allows the drop)
   ↓  overId !== "item-4" → setOverId("item-4"), later calls skip
render: Item 4 gets class "over" (dashed border)
```

```
   ↓  mouse released over Item 4 → drop fires on Item 4
onDrop(e, "item-4")
   ↓  setItems(prev => moveItem(prev, indexOf("item-1") = 0, indexOf("item-4") = 3))
   ↓  setAnnouncement("Item 1 moved to position 4 of 6")
   ↓  resetDrag()
   ↓  dragend fires on Item 1 → resetDrag() again (harmless)
render: order 2 3 4 1 5 6, no extra classes, screen reader reads the announcement
```

7. Important technical challenges

Challenge 1: indexes go stale, identity must be the id

- *Problem:* The first version stored the dragged **index** and used the title as `key`. Scenario: drag starts on index 0; meanwhile the list changes (a card is added/removed, or another reorder lands) and index 0 is now a different card: the wrong card moves. Two items with the same title also break React's reconciliation.
- *Solution:* Every item has a stable `id`. Track `draggingId`, and turn ids into indexes **at drop time** inside the functional updater, so it always uses the latest list. The reorder itself is a pure function you can test in isolation.

```
setItems((prev) =>
  moveItem(prev, prev.findIndex((it) => it.id === sourceId),
                 prev.findIndex((it) => it.id === targetId))
);
```

Challenge 2: making the browser accept the drop

- *Problem:* By default, elements are **not** drop targets. Without `preventDefault()` in `dragover`, `drop` never fires and the cursor shows "not allowed". Firefox also refuses to start a drag unless `dataTransfer` has some data. And `dragenter` / `dragleave` fire again for every child element, so a "hover" flag based on them flickers.
- *Solution:* Call `preventDefault()` in `dragover` (and in `drop`, so Firefox doesn't navigate to the dropped text). Always `setData`. Track the target with `dragover` + an id (not enter/leave counters), and set `pointer-events: none` on the card's children so the card is the only target.

```
const onDragOver = (e, id) => {
  e.preventDefault();               // without this the browser refuses the drop
  e.dataTransfer.dropEffect = "move";
  if (overId !== id) setOverId(id); // dragover fires every ~50ms: skip no-op updates
};
// CSS: .item > * { pointer-events: none; }
```

Challenge 3: cleanup when the drop never happens

- *Problem:* The user drags Item 2, hovers Item 5, then presses Escape or lets go over empty space. `drop` never fires, so if cleanup lives only in `onDrop`, Item 2 stays faded and Item 5 keeps its dashed border forever. The original code had exactly this gap.
- *Solution:* `dragend` always fires on the **source** element after every drag, dropped or cancelled. Put the reset there (and also in `drop`, which is harmless).

```
const resetDrag = () => { setDraggingId(null); setOverId(null); };
<li onDragEnd={resetDrag} ... />
```

Challenge 4: keyboard and screen-reader users

- *Problem:* HTML5 DnD is mouse-only. A keyboard user can't reorder at all, and a screen-reader user gets no feedback. A second issue: when React moves a card's DOM node during a keyboard move, the browser can drop focus, and the user is lost.
- *Solution:* The standard "pick up / move / drop" pattern: Space/Enter toggles `grabbed`, arrows call the same `moveItem`, Escape restores a `snapshot` taken at pick-up. Every move updates an `aria-live` region. A `useEffect` puts focus back on the grabbed card after each move.

```
if (e.key === "Escape") { setItems(grabbed.snapshot); setGrabbed(null); return; }
const step = STEP[e.key];                 // ArrowUp/Left: -1, ArrowDown/Right: +1
const to = index + step;
if (to < 0 || to >= items.length) return; // clamp at both ends
setItems(moveItem(items, index, to));

useEffect(() => {
  if (grabbed) listRef.current.querySelector(`[data-id="${grabbed.id}"]`)?.focus();
}, [items, grabbed]);
```

8. Implement in the correct order

#	Step	Working result
1	Mock data with ids; render cards in a `<ul>` grid	Static list on screen
2	Pure `moveItem` + quick asserts	Reorder logic proven before any UI
3	`draggable` + `onDragStart` (setData, `draggingId`)	Cards can be picked up
4	`onDragOver` with `preventDefault` + `onDrop` calling `moveItem`	Mouse reorder works
5	`onDragEnd` reset	Cancelled drags leave no state
6	`overId` + `dragging` / `over` classes, `pointer-events: none` on children	Visual feedback, no flicker
7	Extract `SortableItem` (presentational)	Clean `App`

#	Step	Working result
8	`tabIndex`, focus ring, Space/Enter pick-up and drop, arrows move	Keyboard reorder
9	Escape restores snapshot, clamp at ends, focus follows the card	Robust keyboard mode
10	`aria-live` region, `aria-label` with position, help text via `aria-describedby`	Screen-reader support
11	Empty state, reduced-motion, responsive container	Polished

9. Cover all states and edge cases

State / edge case	Handling
Initial	Cards in default order, nothing dragged or grabbed
Dragging (mouse)	Source has `dragging` (faded)
Over a target	Target has `over` (dashed border); source never shows `over`
Drop success	`moveItem`, announcement, state reset
Drop on itself	`sourceId === targetId` → no change
Drop outside / Esc mid-drag	`dragend` resets; order unchanged
Drag over the gap between cards	No `preventDefault` there → drop not allowed; last `overId` stays until `dragend`
Grabbed (keyboard)	Card has `grabbed`; arrows move it; focus stays on it
Arrow at first / last position	Clamped, no change
Escape while grabbed	Original order restored from `snapshot`
Tab away while grabbed	Card stays grabbed; next Space/Enter on any card drops it
Empty list	"No items to reorder."
Single item	Drag and arrows are no-ops
Duplicate titles	Fine: keys and logic use ids
Dropping text/files from outside the page	`draggingId` is null; payload isn't a known id → `findIndex` = -1 → `moveItem` returns the list unchanged
Touch device	Native DnD doesn't fire; keyboard still works. Pointer-events version is the upgrade
Reduced motion	Transitions turned off

10. Final structure

```
_21_drag_and_drop_interection/
└── src/
    ├── main.jsx
```

```
    ├── index.css                reset + body
    ├── App.jsx                  state + mouse and keyboard handlers
    ├── App.css                  grid, card states, sr-only
    ├── components/
    │   └── SortableItem.jsx     one presentational card
    ├── data/
    │   └── items.js             INITIAL_ITEMS with ids
    └── utils/
        └── reorder.js           pure moveItem
```

11. Full working code

src/utils/reorder.js

```
// Pure: returns a NEW array with the item at `from` moved to `to`.
export const moveItem = (list, from, to) => {
  if (from === to || from < 0 || to < 0 || from >= list.length || to >= list.length) return list;
  const next = [...list];
  const [moved] = next.splice(from, 1);
  next.splice(to, 0, moved);
  return next;
};
```

src/data/items.js

```
// Stable ids: React keys + drag identity never depend on the position.
export const INITIAL_ITEMS = [
  { id: "item-1", title: "Item 1" },
  { id: "item-2", title: "Item 2" },
  { id: "item-3", title: "Item 3" },
  { id: "item-4", title: "Item 4" },
  { id: "item-5", title: "Item 5" },
  { id: "item-6", title: "Item 6" },
];
```

src/components/SortableItem.jsx

```
const SortableItem = ({ item, position, total, isDragging, isOver, isGrabbed, handlers }) => {
  const className = ["item", isDragging && "dragging", isOver && "over", isGrabbed && "grabbed"]
    .filter(Boolean)
    .join(" ");

  return (
    <li
```

```
      className={className}
      data-id={item.id}
      draggable
      tabIndex={0}
      aria-roledescription="sortable item"
      aria-label={`${item.title}, position ${position} of ${total}`}
      aria-describedby="dnd-help"
      onDragStart={(e) => handlers.onDragStart(e, item.id)}
      onDragOver={(e) => handlers.onDragOver(e, item.id)}
      onDrop={(e) => handlers.onDrop(e, item.id)}
      onDragEnd={handlers.onDragEnd}
      onKeyDown={(e) => handlers.onKeyDown(e, item.id)}
    >
      <span className="handle" aria-hidden="true">⠿</span>
      <p>{item.title}</p>
    </li>
  );
};

export default SortableItem;
```

src/App.jsx

```
import { useEffect, useRef, useState } from "react";
import SortableItem from "./components/SortableItem";
import { INITIAL_ITEMS } from "./data/items";
import { moveItem } from "./utils/reorder";
import "./App.css";

const STEP = { ArrowUp: -1, ArrowLeft: -1, ArrowDown: 1, ArrowRight: 1 };

const App = () => {
  const [items, setItems] = useState(INITIAL_ITEMS);
  const [draggingId, setDraggingId] = useState(null); // mouse drag source
  const [overId, setOverId] = useState(null); // current drop target
  const [grabbed, setGrabbed] = useState(null); // keyboard: { id, snapshot }
  const [announcement, setAnnouncement] = useState("");
  const listRef = useRef(null);

  const indexOf = (id) => items.findIndex((it) => it.id === id);
  const titleOf = (id) => items.find((it) => it.id === id)?.title;

  // Keep focus on the grabbed item after React moves its DOM node.
  useEffect(() => {
    if (grabbed) listRef.current.querySelector(`[data-id="${grabbed.id}"]`)?.focus();
```

```
  }, [items, grabbed]);

  // ---------- mouse (HTML5 Drag and Drop) ----------
  const resetDrag = () => {
    setDraggingId(null);
    setOverId(null);
  };

  const onDragStart = (e, id) => {
    e.dataTransfer.effectAllowed = "move";
    e.dataTransfer.setData("text/plain", id); // Firefox won't start a drag without data
    setDraggingId(id);
  };

  const onDragOver = (e, id) => {
    e.preventDefault(); // without this the browser refuses the drop
    e.dataTransfer.dropEffect = "move";
    if (overId !== id) setOverId(id); // dragover fires every ~50ms: skip no-op updates
  };

  const onDrop = (e, targetId) => {
    e.preventDefault();
    const sourceId = draggingId ?? e.dataTransfer.getData("text/plain");
    if (sourceId && sourceId !== targetId) {
      const to = indexOf(targetId);
      setItems((prev) =>
        moveItem(prev, prev.findIndex((it) => it.id === sourceId), prev.findIndex((it) => it.id ===
targetId))
      );
      setAnnouncement(`${titleOf(sourceId)} moved to position ${to + 1} of ${items.length}`);
    }
    resetDrag();
  };

  // ---------- keyboard ----------
  const onKeyDown = (e, id) => {
    const index = indexOf(id);

    if (e.key === " " || e.key === "Enter") {
      e.preventDefault();
      if (grabbed) {
        setAnnouncement(`${titleOf(grabbed.id)} dropped at position ${indexOf(grabbed.id) + 1}`);
        setGrabbed(null);
      } else {
        setGrabbed({ id, snapshot: items });
        setAnnouncement(`${titleOf(id)} picked up. Arrow keys to move, Space to drop, Escape to
```

```
cancel.`);
      }
      return;
    }

    if (grabbed?.id !== id) return;

    if (e.key === "Escape") {
      setItems(grabbed.snapshot);
      setGrabbed(null);
      setAnnouncement(`Move cancelled. ${titleOf(id)} returned to its place.`);
      return;
    }

    const step = STEP[e.key];
    if (!step) return;
    e.preventDefault(); // stop the page from scrolling
    const to = index + step;
    if (to < 0 || to >= items.length) return;
    setItems(moveItem(items, index, to));
    setAnnouncement(`${titleOf(id)} moved to position ${to + 1} of ${items.length}`);
  };

  const handlers = { onDragStart, onDragOver, onDrop, onDragEnd: resetDrag, onKeyDown };

  return (
    <main>
      <h1>Drag and Drop Interaction</h1>
      <p id="dnd-help" className="help">
        Drag a card to reorder. Keyboard: focus a card, Space to pick up, arrow keys to move, Space to
drop, Escape
        to cancel.
      </p>

      {items.length === 0 ? (
        <p className="empty">No items to reorder.</p>
      ) : (
        <ul className="container" ref={listRef}>
          {items.map((item, index) => (
            <SortableItem
              key={item.id}
              item={item}
              position={index + 1}
              total={items.length}
              isDragging={draggingId === item.id}
              isOver={overId === item.id && draggingId !== item.id}
```

```
              isGrabbed={grabbed?.id === item.id}
              handlers={handlers}
            />
          ))}
        </ul>
      )}

      <p className="sr-only" aria-live="assertive">
        {announcement}
      </p>
    </main>
  );
};

export default App;
```

src/App.css

```
main {
  padding: 20px 16px;
}

h1 {
  text-align: center;
  font-size: 36px;
  font-weight: bold;
  color: white;
  margin-bottom: 12px;
}

.help,
.empty {
  text-align: center;
  color: #bbb;
  margin-bottom: 24px;
}

.container {
  list-style: none;
  max-width: 800px;
  margin: 0 auto;
  padding: 20px;
  background-color: white;
  border-radius: 12px;
  display: flex;
```

```
  flex-wrap: wrap;
  justify-content: center;
  gap: 20px;
}

.item {
  position: relative;
  width: 180px;
  height: 180px;
  padding: 20px;
  border: 2px solid transparent;
  border-radius: 8px;
  background-color: purple;
  color: white;
  cursor: grab;
  user-select: none;
  transition: transform 150ms, opacity 150ms;
}

/* children never become drag targets themselves */
.item > * {
  pointer-events: none;
}

.item:focus-visible {
  outline: 3px solid #f5c518;
  outline-offset: 2px;
}

.item.dragging {
  opacity: 0.4;
}

.item.over {
  border: 2px dashed #f5c518;
  transform: scale(1.04);
}

.item.grabbed {
  cursor: grabbing;
  transform: scale(1.06);
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.35);
}

.handle {
  position: absolute;
```

```
  top: 8px;
  right: 12px;
  font-size: 20px;
  opacity: 0.7;
}

.sr-only {
  position: absolute;
  width: 1px;
  height: 1px;
  overflow: hidden;
  clip: rect(0 0 0 0);
  white-space: nowrap;
}

@media (prefers-reduced-motion: reduce) {
  .item {
    transition: none;
  }
}
```

`src/index.css`

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  -webkit-font-smoothing: antialiased;
  text-rendering: optimizeLegibility;
}

body {
  background-color: black;
  font-family: system-ui, sans-serif;
}
```

`src/main.jsx` (unchanged from the Vite template)

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
```

```
    <App />
  </StrictMode>,
)
```

Quick self-check for the pure logic (`node check.mjs`)

```
import assert from "node:assert";
import { moveItem } from "./src/utils/reorder.js";
const l = ["a","b","c","d"];
assert.deepStrictEqual(moveItem(l,0,2), ["b","c","a","d"]);
assert.deepStrictEqual(moveItem(l,3,0), ["d","a","b","c"]);
assert.deepStrictEqual(moveItem(l,1,2), ["a","c","b","d"]);
assert.strictEqual(moveItem(l,1,1), l);
assert.strictEqual(moveItem(l,-1,2), l);
assert.strictEqual(moveItem(l,1,9), l);
assert.deepStrictEqual(l, ["a","b","c","d"]);
console.log("reorder ok");
```

12. Wrap-up

Edge cases handled

- Drop on itself, drop outside, Escape mid-drag, external drops: no change and no leftover styles.
- Stale indexes and duplicate titles: everything keys off ids, indexes are resolved at drop time.
- Keyboard: pick up, move, clamp at ends, cancel with restore, focus stays on the moving card.
- Screen readers: position in each card's label, every move announced.

Trade-offs to mention

- **Native HTML5 DnD vs a library:** zero dependencies and the browser draws the drag ghost, but no touch support, limited styling of the ghost, and quirks (Firefox `setData`, `dragover` spam). dnd-kit solves touch, auto-scroll, collision detection and animations for a ~10 kB cost.
- **Reorder on drop vs live reorder:** on-drop is simpler and avoids layout jumping under the pointer; live preview (other cards sliding away) feels nicer but needs FLIP animation and hysteresis to avoid flicker.
- **"Take its place" semantics:** dropping on a card puts the dragged card at that index. Dropping *before/after* based on which half of the card the pointer is in is more precise but more code.
- **State in `App` vs a hook:** fine for one list; extract `useSortable` once a second list needs it.

What to add with more time

- Touch / pen support with Pointer Events (`pointerdown` + `setPointerCapture` + `document.elementFromPoint`) or dnd-kit.
- Persist the order (localStorage or `PATCH` API with optimistic update and rollback).
- Drag between multiple lists (Kanban): state becomes `{ [listId]: id[] }`.
- FLIP animation so other cards slide into place; auto-scroll near the container edges.

- Before/after drop indicator based on pointer position.
- Tests: unit tests for `moveItem`; React Testing Library tests firing `dragStart/dragOver/drop/dragEnd` and keyboard events; Playwright for a real mouse drag.

13. Counter questions the interviewer may ask

Requirements

Q1. What if items must move between two lists, like a Kanban board?

Change the state to `{ columns: { todo: ["item-1", ...], done: [...] }, itemsById }`. The drag payload carries `{ id, fromList }`; on drop, remove from the source array and insert into the target array (same list → `moveItem`). Columns themselves become drop targets so an empty column can receive cards.

Q2. Does this work on mobile?

No. Most mobile browsers don't fire HTML5 drag events for touch. Options: Pointer Events (`pointerdown`, `pointermove` with `setPointerCapture`, find the target with `document.elementFromPoint`) or a library like dnd-kit with a touch sensor. The keyboard path still works with an external keyboard.

Q3. Why not just use react-beautiful-dnd or dnd-kit?

In an interview they usually want to see you understand the native events. In production I'd pick dnd-kit: touch, keyboard, auto-scroll and animations are solved and tested. react-beautiful-dnd is deprecated, so I wouldn't start a new project on it.

State design

Q4. Why store `draggingId` instead of the index like the first version?

The index is the very thing that changes during this feature. If the list changes between drag start and drop, the stored index points to a different card. An id always points to the same card; I convert to indexes only at drop time, from the latest list.

Q5. Why is `overId` in state? Couldn't CSS `:hover` do it?

`:hover` doesn't update during a native drag in most browsers, because the pointer is "owned" by the drag operation. So the drop target must be tracked in JS from `dragover`.

Q6. Why keep a `snapshot` for keyboard mode instead of moving back step by step?

Escape should restore the exact order from before pick-up, no matter how many moves happened. A snapshot is one array reference (no copy, since `moveItem` never mutates), and restoring is one `setItems`.

Challenge: making the drop work

Q7. What happens if you remove `e.preventDefault()` from `onDragOver`?

The element isn't a valid drop target: the cursor shows "not allowed" and `drop` never fires. The default action of `dragover` is "reject the drop", so preventing it is how you opt in.

Q8. Why `dragover` and not `dragenter` / `dragleave` for the highlight?

`dragenter` / `dragleave` fire again whenever the pointer crosses a child element (the title `<p>`, the handle), so the highlight flickers on and off. `dragover` on the card plus `pointer-events: none` on children gives one stable target. The alternative is an enter/leave counter per card, which is more state for no gain.

Q9. `dragover` fires every 50 ms. Isn't that a performance problem?

The handler is cheap, and `if (overId !== id)` skips the state update when the target hasn't changed (React would also bail out on an identical value, but the guard avoids even scheduling it). For heavy cards, memoize `SortableItem` with `React.memo` and stable handlers so only the two cards whose flags changed re-render.

Challenge: cleanup

Q10. Why reset in `dragend` and not only in `drop`?

`drop` fires on the target only when the drop is accepted. If the user presses Escape or releases outside a card, only `dragend` fires (on the source). Resetting in `dragend` covers every ending.

Q11. Why read `draggingId` from state instead of `e.dataTransfer.getData`?

State is synchronous and available in every handler; `getData` is only readable inside `drop` (protected mode during `dragover`). I still `setData`, because Firefox needs it to start a drag and it's the fallback for drops coming from another window.

Challenge: accessibility

Q12. How does a screen-reader user know the card moved?

An `aria-live="assertive"` region gets a new sentence on every pick-up, move, drop and cancel ("Item 3 moved to position 1 of 6"). Each card's `aria-label` includes its position, and `aria-describedby` links the instructions. `aria-grabbed` / `aria-dropeffect` are deprecated, so I don't use them.

Q13. Why can focus get lost after a keyboard move, and how do you fix it?

React reorders by moving DOM nodes with `insertBefore`; moving the focused node can blur it in some browsers. Stable keys keep the same node, and a `useEffect` on `[items, grabbed]` refocuses the grabbed card, so focus always follows it.

Implementation steps

Q14. Why is `moveItem` a separate pure function?

It's the only real logic, and as a pure function it's testable with plain asserts, reusable for mouse and keyboard, and safe for React because it returns a new array. It also returns the same reference for no-ops, so React skips the re-render.

Q15. Why use the functional updater `setItems(prev => ...)` in `onDrop`?

It computes indexes from the latest list, even if another update is queued in the same tick. With the closed-over `items` you could compute from a stale list. In the keyboard handler each key press is its own event with

a fresh render, so using `items` directly is fine there.

Q16. How would you show a before/after line instead of "take its place"?

In `dragover`, compare `e.clientY` (or `clientX` in a row) with the target's `getBoundingClientRect()` midpoint and store `{ overId, side: "before" | "after" }`. On drop, compute `to = targetIndex + (side === "after" ? 1 : 0)`, adjusted by -1 when moving down because removal shifts indexes.

Performance / scaling ("what if")

Q17. What if the list has 10,000 items?

Rendering is the problem, not the reorder (one `splice` is O(n), fine). Virtualize the list (react-window), memoize rows, and keep handlers stable with `useCallback` so only the source and target re-render. Auto-scroll the container when dragging near its edges, since the target may be off screen.

Q18. What if the order must be saved to a slow or failing API?

Optimistic update: change the order locally right away, send `PATCH /order` with the id array, and on error restore the previous order and show a toast. Debounce or queue requests so rapid keyboard moves send one request with the final order, and ignore stale responses.

Q19. What if two users reorder the same list at the same time?

Send a version number with the update; the server rejects stale versions and the client refetches. For live collaboration, use fractional indexing (each item has a sortable rank string) so a move only changes one item's rank instead of the whole array, which reduces conflicts.

Trade-offs

Q20. Why not reorder live while hovering instead of on drop?

Live reorder moves the card under the pointer, which changes the target, which moves the card again: flicker unless you add hysteresis and FLIP animations. On-drop is predictable and was enough for the stated requirements; I'd add live preview if the interviewer wants that feel.

Q21. Why is all the state in `App` and not in a custom hook?

One list, one component: a hook would be indirection without reuse. As soon as a second sortable list appears, I'd move the state and handlers into `useSortable(items, setItems)` returning `getItemProps(id)`.